

at every transformer layer. This is akin to learning transformer activations that are fixed across examples at every network layer. In contrast, prompt tuning uses a single prompt representation that is prepended to the embedded input. Beyond requiring fewer parameters, our approach allows the transformer to update the intermediate-layer task representations, as contextualized by an input example. Their work builds on GPT-2 (Radford et al., 2019) and BART (Lewis et al., 2020), while ours focuses on T5 and examines changes in performance and robustness to design choices as model size increases. When using BART, prefix tuning includes prefixes on both the encoder and decoder network, while prompt tuning only requires prompts on the encoder. Li and Liang (2021) also rely on a reparameterization of the prefix to stabilize learning, which adds a large number of parameters during training, whereas our configuration does not require this reparameterization and is robust across SuperGLUE tasks and model sizes.

Hambardzumyan et al. (2021) propose “WARP”, where prompt parameters are added to the input layer. This method works with masked language models, relying on a [MASK] token and a learnable output layer to project the mask to class logits. This formulation restricts the model to producing a single output, limiting it to classification. Prompt tuning does not require any changes to the input or a task-specific head. The performance of prompt tuning is also considerably closer to the strong performance of model tuning.

Liu et al. (2021) propose “P-tuning” where learnable continuous prompts are interleaved throughout the embedded input, using patterns based on human design. Our approach removes this complication by simply prepending the prompt to the input. To achieve strong SuperGLUE results, P-tuning has to be used in *conjunction* with model tuning, that is, models jointly update both the prompt and the main model parameters, whereas our approach keeps the original language model frozen.¹⁰

Qin and Eisner (2021) use “soft words” to learn prompts to extract knowledge from pre-trained LMs. Prompts are positioned in relation to the input based on hand-designed prompt prototypes, and a learned Δ_i^ℓ parameter is included for each layer, so parameter cost scales with model depth.

¹⁰As another difference, P-tuning requires the addition of “anchor” tokens in the input (e.g. a question mark following the hypothesis in the RTE task) to achieve strong performance, while prompt tuning leaves inputs untouched.

Dataset	Domain	Model	Prompt	Δ
SQuAD	Wiki	94.9 $\pm$ 0.2	94.8 $\pm$ 0.1	−0.1
TextbookQA	Book	54.3 $\pm$ 3.7	66.8 $\pm$ 2.9	+12.5
BioASQ	Bio	77.9 $\pm$ 0.4	79.1 $\pm$ 0.3	+1.2
RACE	Exam	59.8 $\pm$ 0.6	60.7 $\pm$ 0.5	+0.9
RE	Wiki	88.4 $\pm$ 0.1	88.8 $\pm$ 0.2	+0.4
DuoRC	Movie	68.9 $\pm$ 0.7	67.7 $\pm$ 1.1	−1.2
DROP	Wiki	68.9 $\pm$ 1.7	67.1 $\pm$ 1.9	−1.8

Table 1: F1 mean and stddev for models trained on SQuAD and evaluated on out-of-domain datasets from the MRQA 2019 shared task. Prompt tuning tends to give stronger zero-shot performance than model tuning, especially on datasets with large domain shifts like TextbookQA.

Logeswaran et al. (2020) use a learnable prepended token to adapt transformer models to various tasks, but focus on small synthetic datasets designed to accommodate a compositional task representation, as opposed to larger real-world datasets. Their base models are small transformers trained from scratch *jointly* with the task representations, whereas we keep the base model frozen and investigate scaling laws using larger transformers.

More generally, work on task prompts is closely aligned with work on “adapters” (Rebuffi et al., 2017; Houlsby et al., 2019), small bottleneck layers inserted *between* frozen pre-trained network layers. Adapters offer another means of reducing task-specific parameters, with Houlsby et al. (2019) achieving GLUE performance close to full model tuning when freezing BERT-Large and only adding 2–4% additional parameters. Pfeiffer et al. (2020) use multiple adapters in a multilingual context to explicitly separate language understanding from task specification, similar to our approach. A core difference between adapters and prompt tuning is how the approaches change model behavior. Adapters modify the actual function that acts on the input representation, parameterized by the neural network, by allowing the rewriting of activations at any given layer. Prompt tuning modifies behavior by leaving the function fixed and adding new input representations that can affect how subsequent input is processed.

5 Resilience to Domain Shift

By freezing the core language model parameters, prompt tuning prevents the model from modifying its general understanding of language. Instead, prompt representations indirectly modulate the representation of the input. This reduces the model’s ability to overfit to a dataset by memorizing spe-

Train	Eval	Tuning	Accuracy	F1
QQP	MRPC	Model Prompt	73.1 $\pm$ 0.9	81.2 $\pm$ 2.1
			76.3 $\pm$ 0.1	84.3 $\pm$ 0.3
MRPC	QQP	Model Prompt	74.9 $\pm$ 1.3	70.9 $\pm$ 1.2
			75.4 $\pm$ 0.8	69.7 $\pm$ 0.3

Table 2: Mean and stddev of zero-shot domain transfer between two paraphrase detection tasks.

cific lexical cues and spurious correlations. This restriction suggests that prompt tuning may improve robustness to domain shifts, where the distribution of inputs differs between training and evaluation.

We investigate zero-shot domain transfer on two tasks: question answering (QA) and paraphrase detection. For question answering, we use the MRQA 2019 shared task on generalization (Fisch et al., 2019). This task collects extractive QA datasets in a unified format and tests how models trained on “in-domain” datasets perform when evaluated on “out-of-domain” datasets. For our experiments, we train on SQuAD (Rajpurkar et al., 2016) and evaluate on each of the out-of-domain datasets.¹¹

Table 1 shows that prompt tuning outperforms model tuning on the majority of out-of-domain datasets, with a remarkable 12.5 point F1 gap between the two approaches on TextbookQA. We observe larger gains from prompt tuning in cases of larger domain shifts (e.g. to Biomedical in BioASQ or to Textbooks in TextbookQA). Of the datasets where model tuning is better, we see that DROP shares a domain (Wikipedia) with SQuAD and is thus one of the smallest domain transfers.

As a second test of robustness to domain shift, we explore transfer between two paraphrase detection tasks from GLUE (Wang et al., 2019b). The first task is QQP (Iyer et al., 2017), which asks if two questions from the community Q&A site Quora are “duplicates”. The second task is MRPC (Dolan and Brockett, 2005), which asks if two sentences drawn from news articles are paraphrases. We test transfer in both directions (QQP $\Leftrightarrow$ MRPC). As before, we train on the “in-domain” task, select checkpoints using in-domain validation, and evaluate zero-shot on the “out-of-domain” task.

Table 2 shows that training a lightweight prompt on the QQP data and evaluating on MRPC gives much better performance than tuning the entire

¹¹We select checkpoints based on SQuAD validation F1. The out-of-domain datasets are TextbookQA (Kembhavi et al., 2017), RACE (Lai et al., 2017), BioASQ (<http://bioasq.org/>), RE (Levy et al., 2017), DuoRC (Saha et al., 2018), and DROP (Dua et al., 2019).

Dataset	Metric	Average	Best	Ensemble
BoolQ	acc.	91.1	91.3	91.7
CB	acc./F1	99.3 / 99.0	100.00 / 100.00	100.0 / 100.0
COPA	acc.	98.8	100.0	100.0
MultiRC	EM/F1 _a	65.7 / 88.7	66.3 / 89.0	67.1 / 89.4
ReCoRD	EM/F1	92.7 / 93.4	92.9 / 93.5	93.2 / 93.9
RTE	acc.	92.6	93.5	93.5
WiC	acc.	76.2	76.6	77.4
WSC	acc.	95.8	96.2	96.2
SuperGLUE (dev)		90.5	91.0	91.3

Table 3: Performance of a five-prompt ensemble built from a single frozen T5-XXL model exceeds both the average and the best among the five prompts.

model (+3.2 accuracy and +3.1 F1). The results are much closer in the other direction, with prompt tuning showing a small improvement in accuracy and a small drop in F1. These results support the view that model tuning may be over-parameterized and more prone to overfit the training task, to the detriment of similar tasks in different domains.

6 Prompt Ensembling

Ensembles of neural models trained from different initializations on the same data are widely observed to improve task performance (Hansen and Salamon, 1990) and are useful for estimating model uncertainty (Lakshminarayanan et al., 2017). However, as model size increases, ensembling can become impractical. Beyond the space required to store N models (e.g. 42 GiB for each copy of T5-XXL), there is a substantial inference cost to running N distinct models, whether in parallel or in series.

Prompt tuning provides a more efficient way to ensemble multiple adaptations of a pre-trained language model. By training N prompts on the same task, we create N separate “models” for a task, while still sharing the core language modeling parameters throughout. Beyond drastically reducing storage costs, the prompt ensemble makes inference more efficient. To process one example, rather than computing forward passes of N different models, we can execute a single forward pass with a batch size of N , replicating the example across the batch and varying the prompt. These savings mirror those seen for multi-tasking in Figure 2.

To demonstrate the viability of prompt ensembling, we train five prompts for each SuperGLUE task, using a single frozen T5-XXL model with our default hyperparameters. We use simple majority voting to compute predictions from the ensemble. Table 3 shows that across all tasks, the ensemble beats the single-prompt average and beats, or

matches, the best individual prompt.

7 Interpretability

An ideally interpretable prompt would consist of natural language that clearly describes the task at hand, explicitly asks the model for some result or action, and makes it easy to understand why the prompt elicited such behavior from the model.

As prompt tuning works in the continuous embedding space rather than the discrete token space, interpreting prompts becomes more difficult. To test the interpretability of our learned soft prompts, we compute the nearest neighbors to each prompt token from the frozen model’s vocabulary. We use cosine distance between the vocabulary embedding vector and the prompt token representation as the similarity metric.

We observe that for a given learned prompt token, the top-5 nearest neighbors form tight semantic clusters. For example, we see lexically similar clusters such as { *Technology* / *technology* / *Technologies* / *technological* / *technologies* }, as well as more diverse but still strongly related clusters such as { *entirely* / *completely* / *totally* / *altogether* / *100%* }. The nature of these clusters suggests that the prompts are in fact learning “word-like” representations. We found that random vectors drawn from the embedding space do not show this sort of semantic clustering.

When initializing the prompts using the “class-label” strategy, we often find that the class labels persist through training. Specifically, if a prompt token is initialized to a given label, that label is often among the learned token’s nearest neighbors after tuning. When initializing with the “Random Uniform” or “Sampled Vocab” methods, the class labels can also be found in the nearest neighbors of the prompts; however they tend to appear as neighbors to multiple prompt tokens. This suggests that the model is learning to store the expected output classes in the prompts as reference, and initializing the prompt to outputs classes makes this easier and more centralized.

When examining longer prompts (e.g. size 100), we often find several prompt tokens with the same nearest neighbors. This suggests there is either excess capacity in the prompt, or that the lack of sequential structure in the prompt representation makes it difficult for the model to localize information to a specific position.

While the learned prompts taken as sequences

show little interpretability, we do observe a high frequency of words like *science*, *technology* and *engineering* as the nearest neighbors for prompts trained on the BoolQ dataset and approximately 20% of the questions are in the “Nature/Science” category. While more investigation is needed, this suggests that one role of the prompt may be to prime the model to interpret inputs in a specific domain or context (e.g. “scientific”).

8 Conclusion

In this paper, we showed that prompt tuning is a competitive technique for adapting frozen pre-trained language models to downstream tasks. On the popular SuperGLUE benchmark, its task performance rivals that of traditional model tuning, with the gap vanishing as model size increases. On zero-shot domain transfer, we found that prompt tuning leads to improved generalization. This plausibly indicates that freezing general-purpose language understanding parameters and restricting downstream learning to a lightweight parameter footprint can help to avoid overfitting to a specific domain.

Beyond task quality metrics, we discussed the appeal of moving to frozen pre-trained models in terms of storage and serving costs. This move enables both efficient multi-task serving, as well as efficient high-performing prompt ensembling. Looking forward, we believe that factoring out task-defining parameters as distinct from general language-modeling parameters is an exciting step that opens up many avenues for new research.

Acknowledgements

We thank Lucas Dixon, Waleed Ammar, Slav Petrov and Sebastian Ruder for comments on an earlier draft, and the following people for helpful discussion: Colin Raffel, Adam Roberts, and Noam Shazeer. We thank Linting Xue for help with the LM adaptation training.

References

- Roy Bar-Haim, Ido Dagan, Bill Dolan, Lisa Ferro, Danilo Giampiccolo, Bernardo Magnini, and Idan Szpektor. 2006. The second PASCAL recognising textual entailment challenge. In *Proceedings of the second PASCAL challenges workshop on recognising textual entailment*, volume 6, pages 6–4. Venice.
- Luisa Bentivogli, Peter Clark, Ido Dagan, and Danilo Giampiccolo. 2009. The fifth PASCAL recognizing textual entailment challenge. In *TAC*.